

Liste des fichiers modifiés :

- NetworkStack.py
- mainComplete.py

Modifications de NetworkStack.py et fonctionnement anneau : On suit le voyage d'un paquet

1- Modification des attributs

```
self.maxMessages = 3 # L'indice du dernier message par paquet
self.nbMessages = 0 # Le nombre actuel de messages traité dans le paquet
self.paquet = bytearray([]) # Le paquet
self.aEcrit = False # Permet que de n'écrire qu'un message par ordinateur par paquet
self.numberOfNodesPerRing = 4 # Nombre de pc par anneau
self.enTraitement = False # Détermine si est en train de traiter un paquet
```

2 – Modification des couches

Layer 2 in : Traite le multiplexage en séparant les fragments (mini-paquets) du paquet grâce à « size », et les envoie à la couche supérieure un par un → **layer2bis in**

Layer 2bis in : Traite si le paquet est plein, si oui, on l'envoie à la couche supérieure, si non, on l'envoie → **application_layer_out**

Layer 3 in : On traite ici l'émetteur, le destinataire, l'accusé de réception et le timeout

Si on est le destinataire:

Si le message est un accusé de réception :

On renvoie un Token → **application_layer_out(forceToken)**

Sinon :

On envoie un accusé de réception → **layer3out**

On amène le message à l'application → **layer4in**

Sinon :

Si le paquet est depuis assez peu de temps sur le réseau et que nous ne l'avons pas envoyé (ainsi il n'a pas fait le tour de l'anneau):

On diminue sa durée de vie

On le renvoie tel quel → **layer3out**

Sinon :

On l'envoie → **application_layer_out**

Layer 4 in : On extrait l'applicationPort et on le passe à la couche supérieure → **application_layer_in**

Application_layer_in : On extrait le message et on le passe à l'application

Application_layer_out :

Si On a déjà écrit, qu'on a ForceToken, ou qu'on a pas de message à envoyer

On met plein à 0

On met destination à ''

On encapsule TOKEN dans le sdu

Sinon

On met plein à 1

On obtient à destination

On encapsule un message dans le sdu

On passe le sdu, plein, la destination et l'applicationPort à la couche inférieure →

Layer4out

Layer4out :

On encapsule l'applicationPort dans le sdu

On met ack à 0 : si on est ici, c'est forcément qu'on a extrait un message de la pile, donc ce n'est pas un ack

On met le nombre de sauts à 0 : même raison

On passe le sdu, plein, la destination, nombre de sauts et ack → **Layer3out**

Layer3out :

On encapsule ack, destination, source, le nombre de sauts dans le sdu

On met l'interface à 0:elle est toujours à 0, on a pas géré différents anneaux

On passe le sdu, plein, → **Layer2bis out**

Layer2bis out :

On encapsule plein

On passe le sdu → **Layer2bis**

Layer2 out : On attend ici de recevoir le nombre de fragments (mini-paquets) et on envoie le paquet

3- Modification d'autres méthodes

LeaveNetwork : Pour la mort d'une instance de computer, on attend que l'attribut enTraitement soit à False

Modifications de mainComplete.py

Ajout des variables globales :

Si networkStack = 0 on utilise networkStack pour les instances de computer, **Sinon** on utilise networkStackAlternative

delay : le temps pour lequel les interfaces vont sleep

Si pcTrois = True il y a un autre pc sur le même anneau, **Sinon** il n'y en pas

Si mortPcTrois = True, l'instance de computer meurt au bout de dureeVie secondes

dureeVie : temps de vie du pcTrois